

ROBOT Z ESP32CAM

TEMAT I CELE ZAJĘĆ

Celem zajęć jest zaprogramowanie robota Sharky do podążania linii, przejechania całego toru oraz zczytywania znaków drogowych za pomocą kamery esp32.

KRÓTKIE UZASADNIENIE DYDAKTYCZNE

Przeprowadzenie tego cyklu lekcji pozwoli uczniom w praktyczny sposób poszerzyć swoje zdolności programistyczne w środowisku Arduino IDE, opartym na języku C++. Zajęcia te pozwalają zrozumieć podstawy działania elektroniki i wdrażania sztucznej inteligencji (AI) w urządzeniach fizycznych.

OPIS PRZEBIEGU ZAJĘĆ

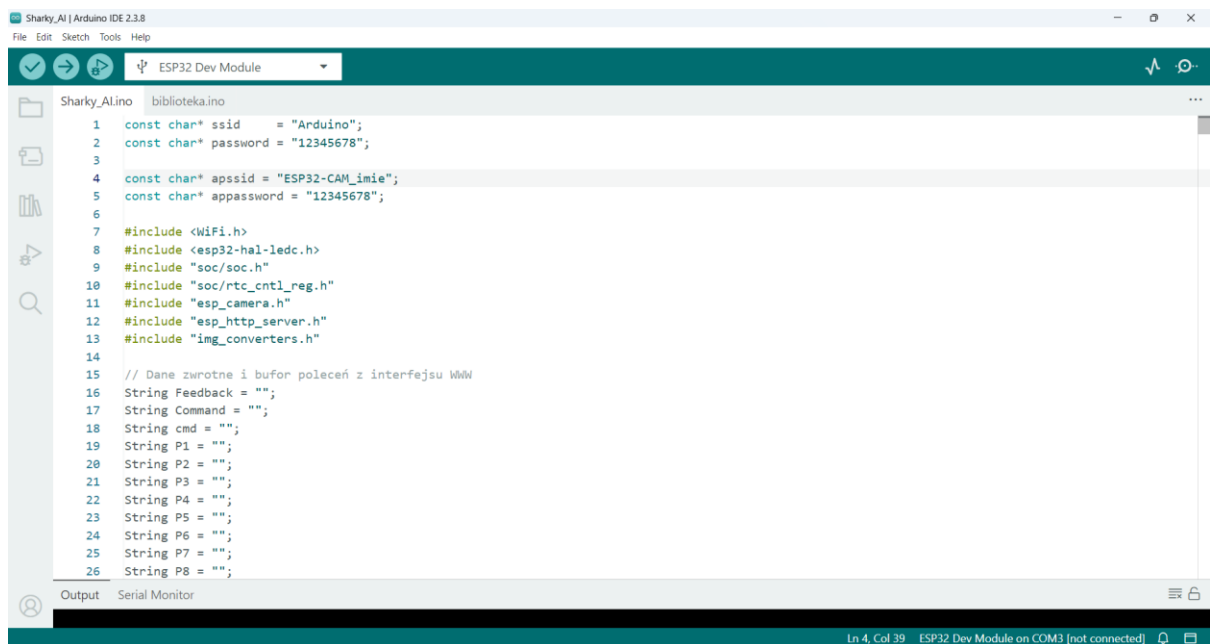
Uczniowie najpierw przerabiają scenariusze ze strony start.sic.edu.pl, aby zrozumieć podstawy działania robota. Następnie na podstawie poniższego scenariusza uczą się trenowania modeli do rozpoznawania klas.

INSTRUKCJA

WSTĘP

Otwórz nasz kod w swoim środowisku Arduino IDE.

Jest to długi na 1000 linii kod. Chociaż wygląda strasznie, my skupimy się jedynie na wybranych fragmentach, które będą nas najbardziej interesowały. Zaczniemy od początku.



```
1 const char* ssid = "Arduino";
2 const char* password = "12345678";
3
4 const char* apssid = "ESP32-CAM_imie";
5 const char* appassword = "12345678";
6
7 #include <WiFi.h>
8 #include <esp32-hal-ledc.h>
9 #include "soc/soc.h"
10 #include "soc/rtc_cntl_reg.h"
11 #include "esp_camera.h"
12 #include "esp_http_server.h"
13 #include "img_converters.h"
14
15 // Dane zwrotne i bufor poleceń z interfejsu WWW
16 String Feedback = "";
17 String Command = "";
18 String cmd = "";
19 String P1 = "";
20 String P2 = "";
21 String P3 = "";
22 String P4 = "";
23 String P5 = "";
24 String P6 = "";
25 String P7 = "";
26 String P8 = "";
```

Linie 1-5: Konfiguracja sieci Wi-Fi

Kod przygotowuje urządzenie do pracy w sieci na dwa sposoby:

- **ssid i password:** Dane do połączenia ESP32 z istniejącą, domową siecią Wi-Fi (w tym przypadku o nazwie „Arduino”).
- **apssid i appassword:** Dane do uruchomienia przez ESP32 własnego punktu dostępowego (Access Point) o nazwie „ESP32-CAM_imie”. Pozwala to na bezpośrednie połączenie się z urządzeniem (np. smartfonem) bez pośrednictwa routera.

Nas będzie interesowało ssid, gdyż praca nad robotem będzie odbywała się za pomocą strony internetowej. Gdybyśmy jednak chcieli sterować robotem manualnie, np. z pomocą telefonu, użylibyśmy apssid. Warto jednak zastąpić słowo “imie” własnym imieniem, aby przy wyszukiwaniu sieci robota nie dochodziło do pomyłek z innymi robotami.

Linie 7-26: biblioteki i zmienne (nie istotne do zrozumienia)

Linia 43: Deklaracja detektora linii i przypisanie mu wartości 0

```

73  #define do_przodu_lewy 2    //funkcje do własnego kodu
74  #define do_tyłu_lewy 14
75  #define do_przodu_prawy 12
76  #define do_tyłu_prawy 13
77
78  int przerwa11 = 50;
79  int previousTime1 = 0;
80  unsigned long currentTime = 0;
81
82
83  void setup() {
84
85      // Wyłączenie resetu przy brownout
86      WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
87
88      // Start portu szeregowego i logów diagnostycznych
89      Serial.begin(115200);
90      Serial.setDebugOutput(true);
91      Serial.println();
92

```

Linie 73-76: Zastąpienie numeru pinu opisem i funkcją silnika. Np zamiast pisać `LedcWrite(14,0);` napiszemy `LedcWrite(do_tyłu_lewy, 0);`. Działa podobnie jak “znajdź i zamień” w wordzie. Wpływa na czytelność kodu. Możemy ustawiać własne zamienniki.

Linie 78-80: Ważne parametry, o nich później

Linia 83: Początek funkcji `setup`, służy do wstępnej konfiguracji peryferiów sprzętowych. Wykonuje się tylko raz.

Linia 89: **`Serial.begin(115200);`** – Uruchamia komunikację z komputerem przez port USB z prędkością 115200 bodów. Pozwala to na podgląd działania programu w oknie *Monitorze Portu Szeregowego*

Linia 91: **`Serial.println();`** = enter (przejdźcie do nowej linii w Monitorze Portu Szeregowego). Gdyby w nawiasie był tekst w cudzysłowie lub zmienna, to w Monitorze Portu Szeregowego najpierw zostanie napisany tekst, a potem enter (na przykład linie 154-156).

```
213
214
215 initRobot();
216
217
218 void loop() {
219   czujnik_p = digitalRead(15);
220
221 }
222
223
224
225 // Funkcja pomocnicza do wysyłania JPEG-a kawałkami
226 static size_t jpg_encode_stream(void * arg, size_t index, const void* data, size_t len) {
227   jpg_chunking_t *j = (jpg_chunking_t *)arg;
228   if (!index) {
229     j->len = 0;
230   }
231   if (httpd_resp_send_chunk(j->req, (const char *)data, len) != ESP_OK) {
232     return 0;
233   }
234   j->len += len;
235   return len;
236 }
237
238 // Obsługa pojedynczego zdjęcia
```

Output Serial Monitor X

Not connected. Select a board and a port to connect automatically.

Ln 215, Col 15 ESP32 Dev Module on COM3 [not connected]

Linia 219: Wywołanie funkcji `initRobot()` z naszej biblioteki (o której za chwilę), inicjującej aktywację pinów do sterowania silnikiem.

Linie 223-225: Funkcja `loop`, wykonująca się w nieskończoność do zamknięcia programu. Program nieustannie sprawdza stan cyfrowy (0 = biała linia lub 1 = czarna linia) na **pinie numer 15** procesora i zapisuje go do zmiennej `czujnik_p`

Sharky_AI.ino

biblioteka.ino

```
401 // Reakcje wyjść PWM na klasy rozpoznane przez model
402 if (P1 == "Class%201") {
403     move_Foward(183);
404 }
405 else if (P1 == "Class%202") {
406     TurnRight(183);
407 }
408 else if (P1 == "Class%203") {
409     TurnLeft(183);
410 }
411 else if (P1 == "Class%204") {
412     if (currentTime - previousTime1 >= przerwa11){
413         previousTime1 = currentTime;
414         TurnHardRight(183);}
415 }
416
417 else if (P1 == "Class%205") {
418     if (currentTime - previousTime1 >= przerwa11){
419         previousTime1 = currentTime;
420         TurnMidiumRight(183);
421     }
422 }
423 else if (P1 == "Class%206"){
424     if (currentTime - previousTime1 >= przerwa11){
425         previousTime1 = currentTime;
426         TurnMidiumLeft(183);
427     }
428 }
```

Linie 401-428: **Najważniejsze linie w kodzie.** Je jako jedyne będziemy modyfikowali. Służą one do rozpoznawania klas na podstawie zdjęć z kamery robota. Czym są klasy zajmiemy się później. Narazie przyjmijmy, że są to kategorie, stworzone na podstawie wcześniejszych zdjęć i stworzonym modelu AI. Jeżeli robot/program rozpozna daną klasę, to wykona jakieś polecenie. Używana jest tutaj funkcja if. Funkcja **if** składa się z warunku w nawiasie okrągłym, który musi zostać spełniony oraz egzekucji kodu w nawiasie klamrowym. Można dodawać kolejne warunki i egzekucje poprzez funkcję **else if**.

Omówmy to na naszym konkretnym przypadku. Jeżeli zostanie rozpoznana klasa 1, to zostanie wykonane funkcja move_Foward(183) (jazda prosto) Wartość w nawiasie to parametr, w tym przypadku to prędkość. Prędkość w nawiasie powinna być w zakresie 180-210. Jeżeli zostanie wykryta klasa 2 to zostanie wykonana funkcja TurnRight(183) (skręt w prawo). Natomiast na przykład, aby została wykonana klasa 6 (skręt w lewo),

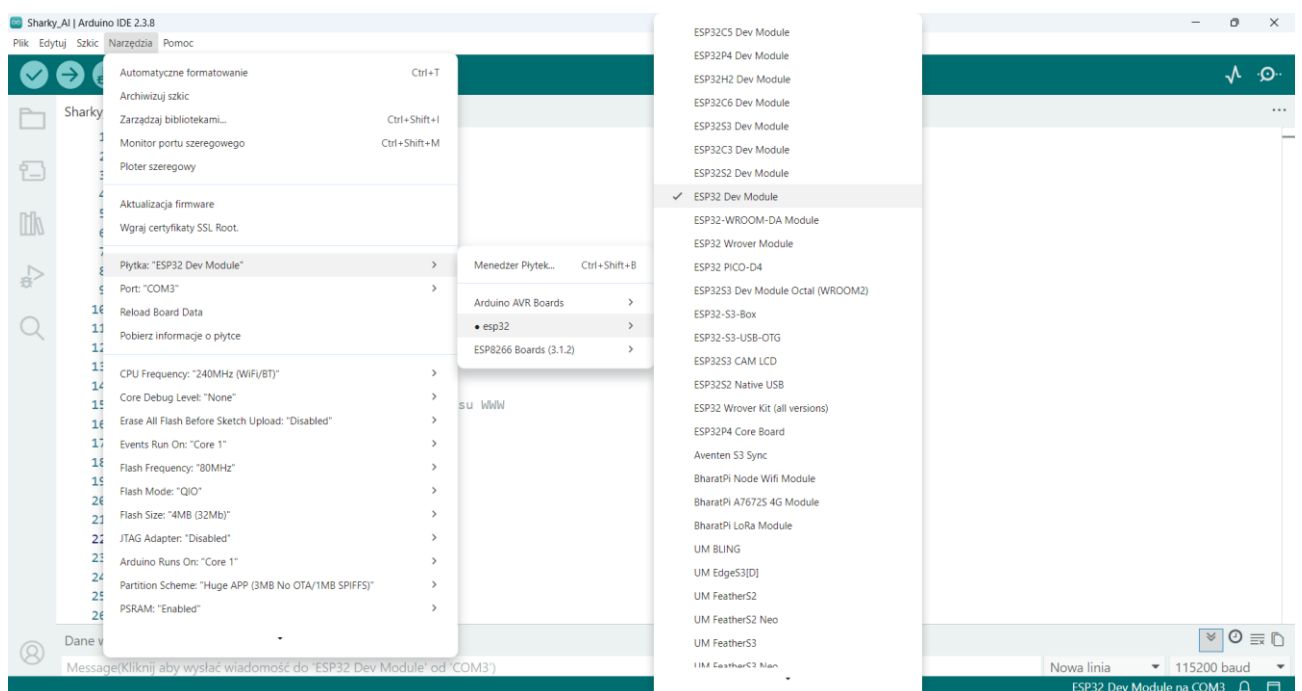
musi dodatkowo być spełniony warunek: `currentTime - previousTime1 >= przerwa11`. Warunek ten działa podobnie jak funkcja `delay()`. Ma to zapobiegać zbyt częstym wykonywaniu się funkcji oraz zacinaniu się kamery. Długość przerwy, `przerwa11`, może być edytowana w 786smej linijce. Obecnie ma wartość 50ms.

Linie 563-999: strona internetowa oparta na ssid

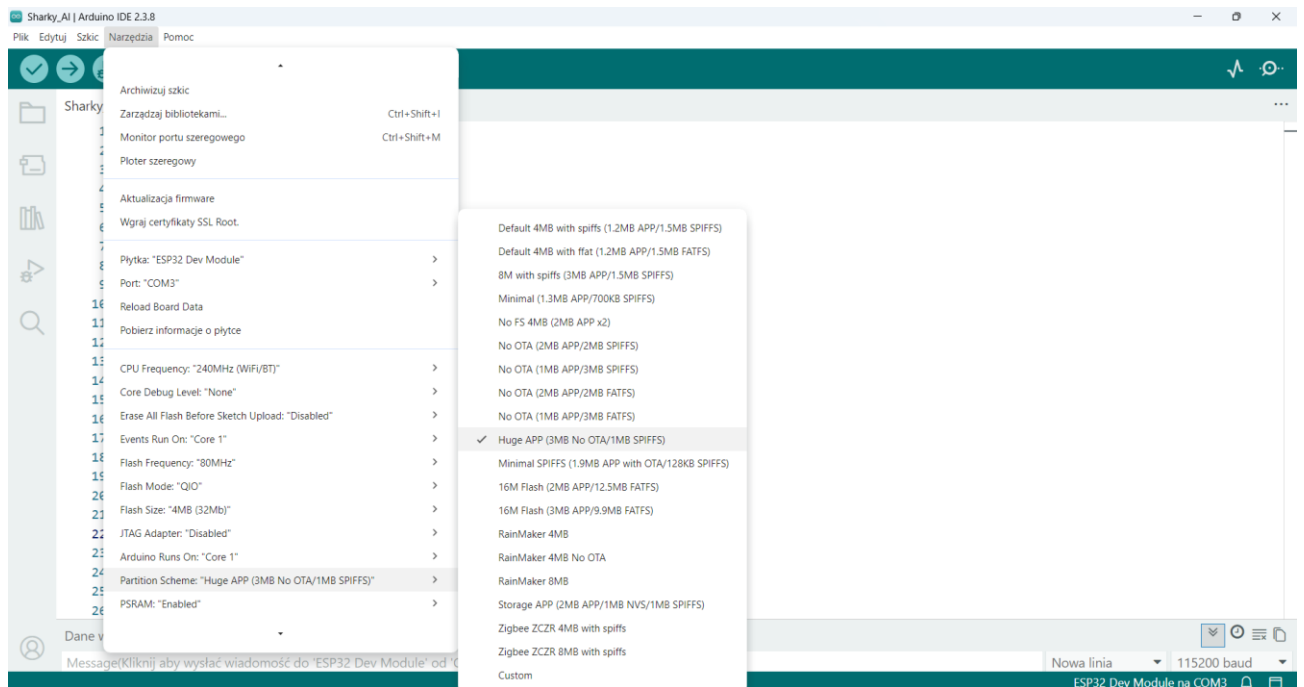
W bibliotece `biblioteka.ino` znajdują się wszystkie zaimplementowane przez nas funkcje. Są one wywoływane w kodzie głównym.

Kod już wstępnie mamy ustalony, to możemy zająć się ustawieniem parametrów, aby wgrać nasz kod na płytkę

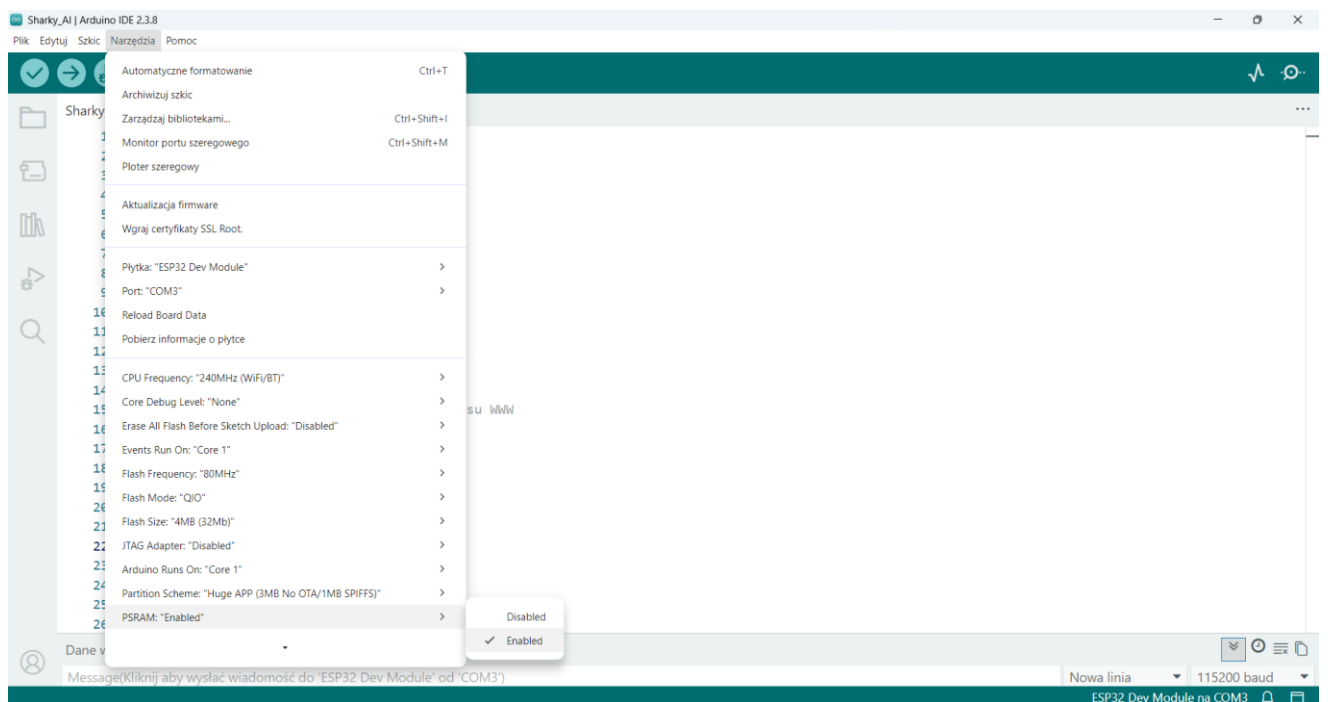
1) Wybierz płytkę ESP32 Dev Module:



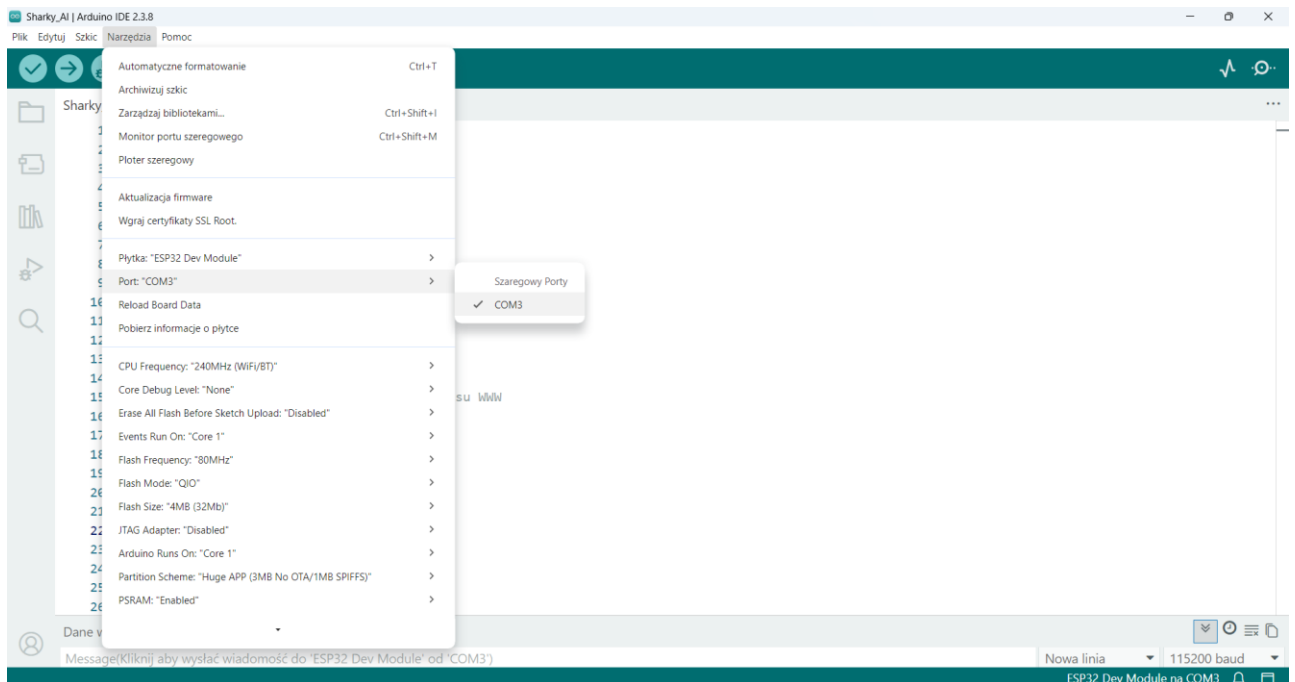
2) Wybierz Partition Scheme na "Huge APP":



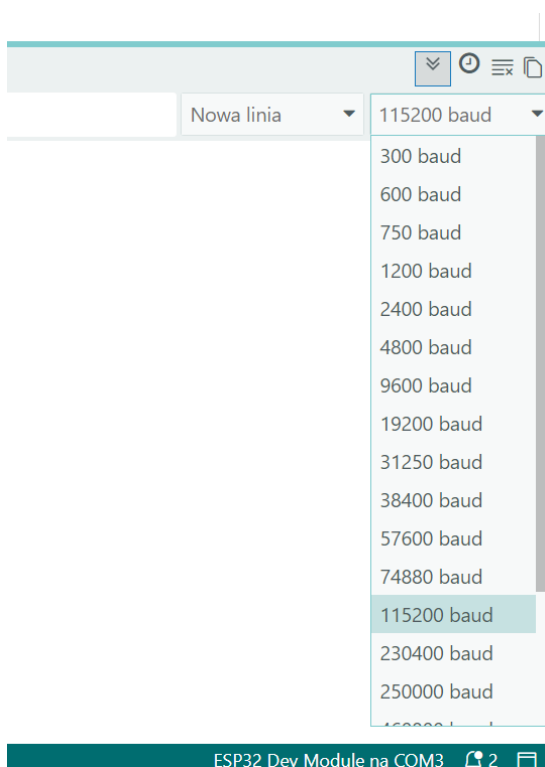
3) Wybierz PSRAM na "Enabled":



4) Wybierz odpowiedni port COM (w tym przypadku jest to COM3, może być inna cyfra):



5) Rozwiń *Monitor Portu Szeregowego*, kliknij dolne, skrajnie prawe pole i wybierz prędkość przesyłu danych 115200 baud



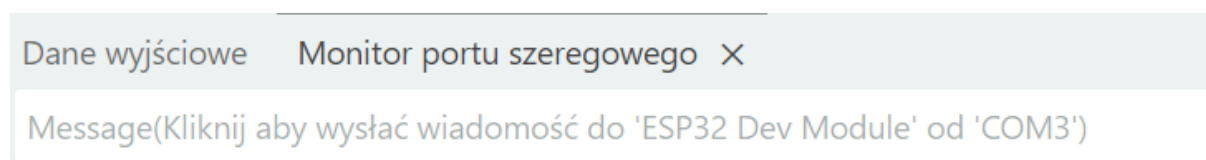
Teraz możesz wgrać program (pamiętaj o odpięciu pinów do detektora linii na czas wgrywania kodu)

harky_AI | Arduino IDE 2.3.8

Edytuj Szkic Narzędzia Pomoc



Po wgraniu kodu otwórz Monitor Portu Szeregowego. Zapewne wygląda on tak:



Connecting to Arduino

STAIP address:
192.168.1.105

APIP address:
192.168.4.1

Starting web server on port: '80'
Starting stream server on port: '81'

Nas interesuje adres STAIP. Może on się różnić dla każdego urządzenia. Po wklejeniu go w przeglądarkę pokaże nam obraz z kamery.

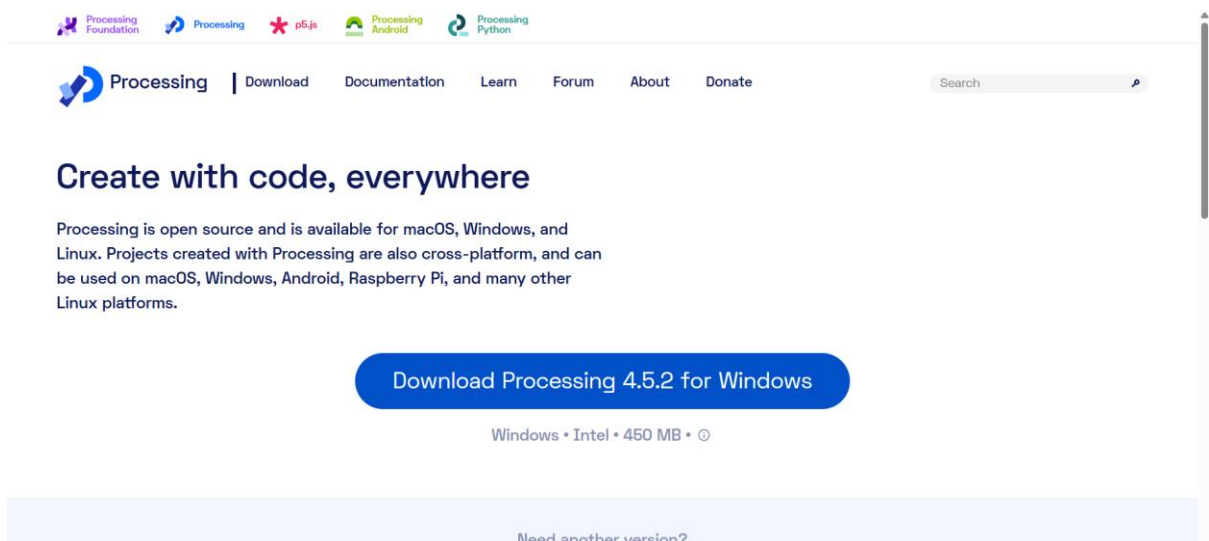


Dlaczego więc widzimy czarne tło z opcjami, a nie obraz z kamery? Otóż użycie naszej kamery nie zostało jeszcze zainicjowane. Potrzebujemy do tego osobnego kodu. W tym celu będziemy musieli jednak opuścić środowisko Arduino i przejść na MIT Processing.

Do czego tak właściwie służy MIT Processing?

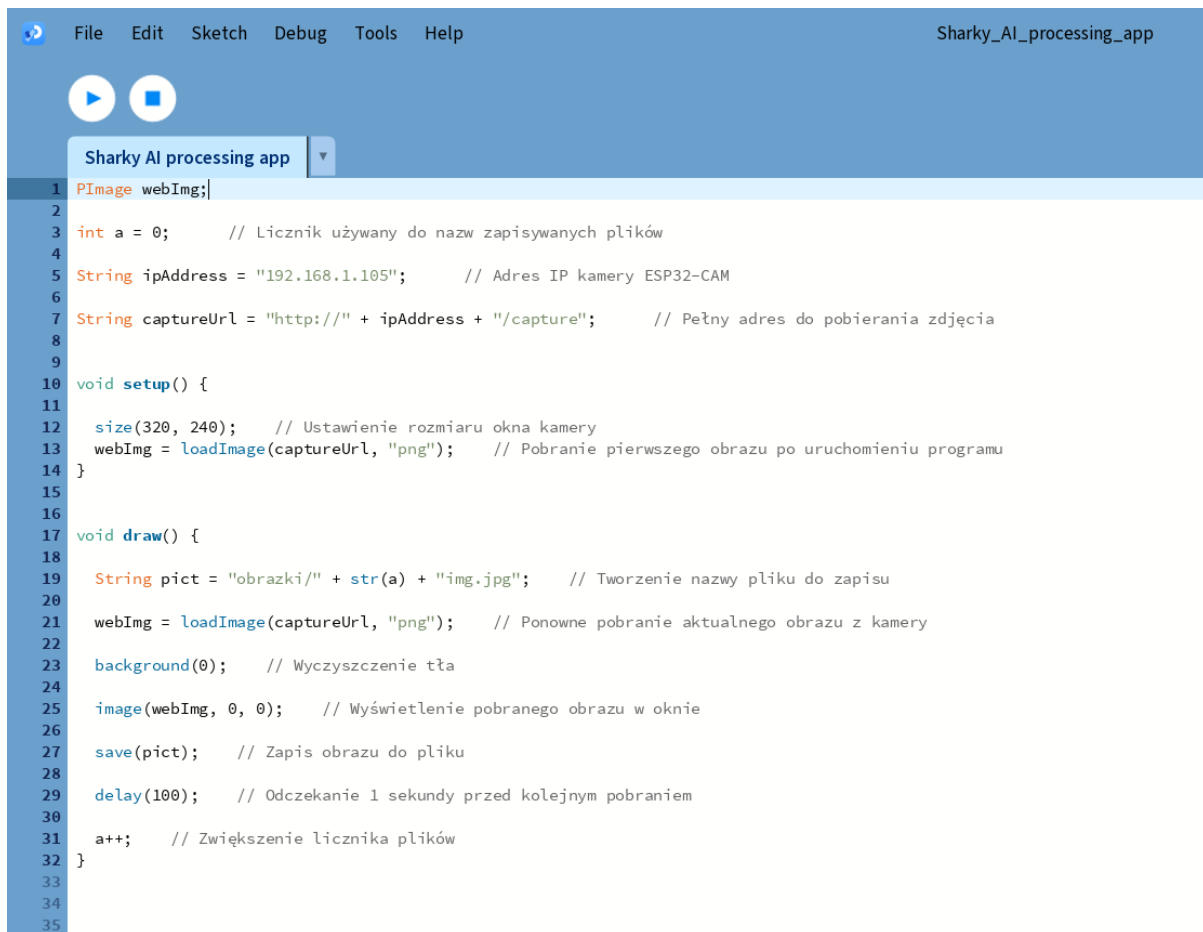
Kamera ESP32 podczas działania rejestruje widziany obraz jako wiele zdjęć robionych w bardzo krótkich odstępach czasu. Dzięki MIT Processing będziemy mogli przechwycić te zdjęcia i zapisać je w pamięci komputera jako obrazy png. Następnie obrazy te będziemy mogli przestać dalej, aby zacząć uczyć sztuczną inteligencję! (o czym później). Ponadto w MIT Processing będziemy mogli włączać i wyłączać pracę kamery.

Wejdź na stronę <https://processing.org/download> i pobierz wersję na Windowsa (proces pobierania analogiczny jak dla Arduino IDE 2.3.8).



Gdy już pobierzesz program, otwórz go. Prawdopodobnie wyskoczą dwa okna, jedno ze stroną MIT Processing, a drugie ze właściwe na wpisanie kodu. Zamknij to pierwsze, następnie skopiuj nasz kod do MIT Processing

Wygląda on tak:

The screenshot shows the Arduino IDE interface. The title bar at the top reads 'Sharky_AI_processing_app'. Below the menu bar (File, Edit, Sketch, Debug, Tools, Help), there are two circular buttons: a play button and a square button. A dropdown menu is open, showing 'Sharky AI processing app'. The main text area contains C++ code for an Arduino sketch. The code includes variable declarations for an image, an integer counter, and strings for IP address and URL. It defines setup() and draw() functions. The setup function sets window size and loads the first image. The draw function generates a filename, loads a new image, clears the background, displays the image, saves it to a file, delays for 100ms, and increments the counter. Line numbers 1 through 35 are visible on the left margin.

```
1 PImage webImg;
2
3 int a = 0; // Licznik używany do nazw zapisywanych plików
4
5 String ipAddress = "192.168.1.105"; // Adres IP kamery ESP32-CAM
6
7 String captureUrl = "http://" + ipAddress + "/capture"; // Pełny adres do pobierania zdjęcia
8
9
10 void setup() {
11
12   size(320, 240); // Ustawienie rozmiaru okna kamery
13   webImg = loadImage(captureUrl, "png"); // Pobranie pierwszego obrazu po uruchomieniu programu
14 }
15
16
17 void draw() {
18
19   String pict = "obrazki/" + str(a) + "img.jpg"; // Tworzenie nazwy pliku do zapisu
20
21   webImg = loadImage(captureUrl, "png"); // Ponowne pobranie aktualnego obrazu z kamery
22
23   background(0); // Wyczyszczenie tła
24
25   image(webImg, 0, 0); // Wyświetlenie pobranego obrazu w oknie
26
27   save(pict); // Zapis obrazu do pliku
28
29   delay(100); // Odczekanie 1 sekundy przed kolejnym pobraniem
30
31   a++; // Zwiększenie licznika plików
32 }
33
34
35
```

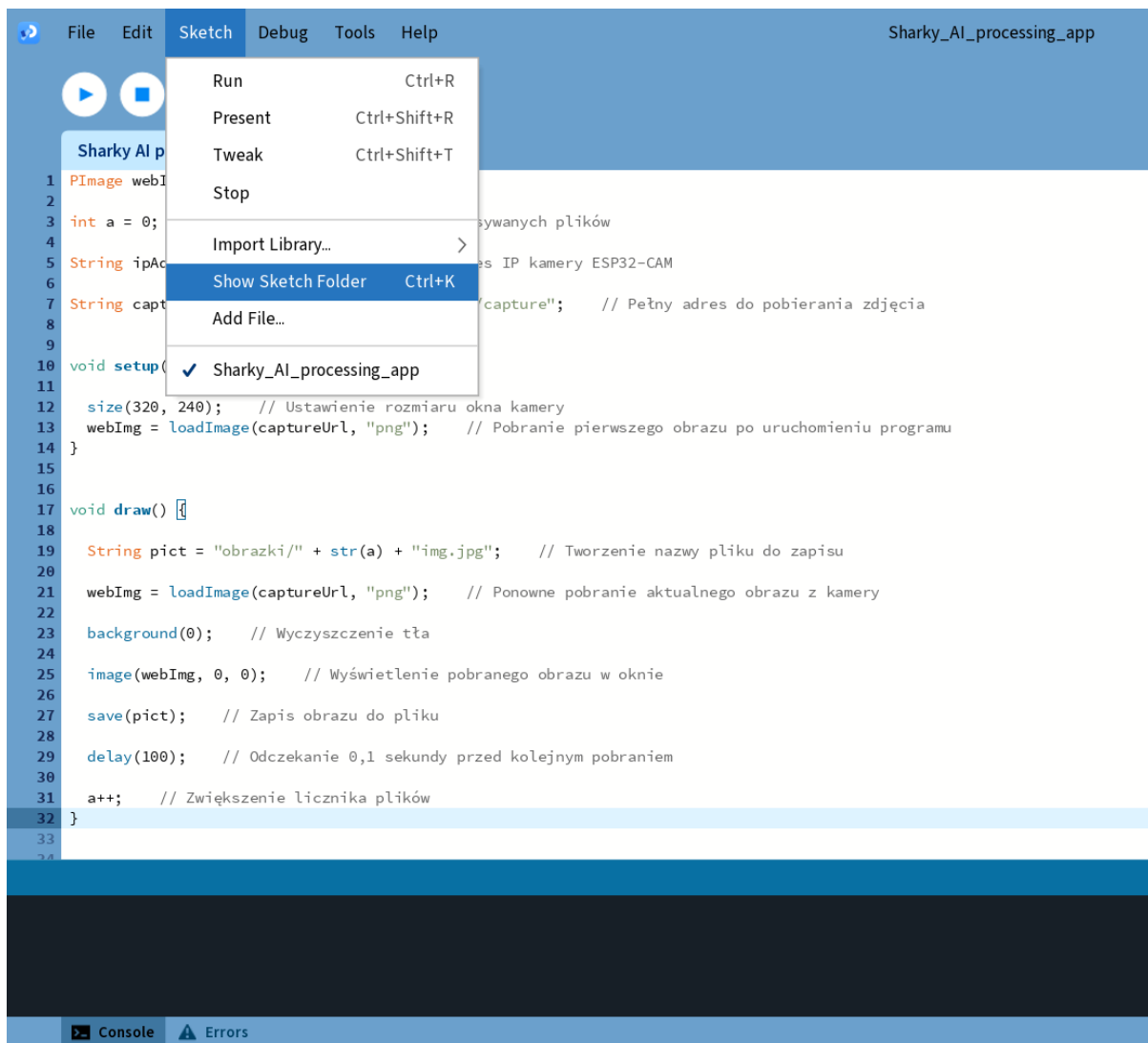
Wszystko jest ładnie pokomentowane, więc zapraszamy do samodzielnego przyswojenia się z kodem. Jedyne rzeczy, na które chcielibyśmy zwrócić uwagę to:

String ipAddress = "192.168.1.105"; - musi być on TAKI SAM jak adres STAIP wygenerowany w *Monitorze Portu Szeregowego*

String pict = "obrazki/" + str(a) + "img.jpg"; - do tego folderu będą zapisywane przechwytywane zdjęcia. Folder zostanie stworzony podczas pierwszej kompilacji programu i do niego będą zapisywane zdjęcia.

Aby rozpocząć nagrywanie obrazu należy nacisnąć trójkąt w lewym górnym rogu ekranu. Pojawi się wtedy małe okienko z widokiem z naszej kamery esp32! Jeżeli chcemy przestać nagrywać musimy nacisnąć kwadrat. Należy pamiętać, że widziany obraz to tak naprawdę dziesiątki zdjęć wykonywanych w czasie pracy kamery. Są one zapisywane do wspomnianego folderu.

Jak dostać się do wspomnianego folderu? Wejść w **Sketch -> Show Sketch Folder**.

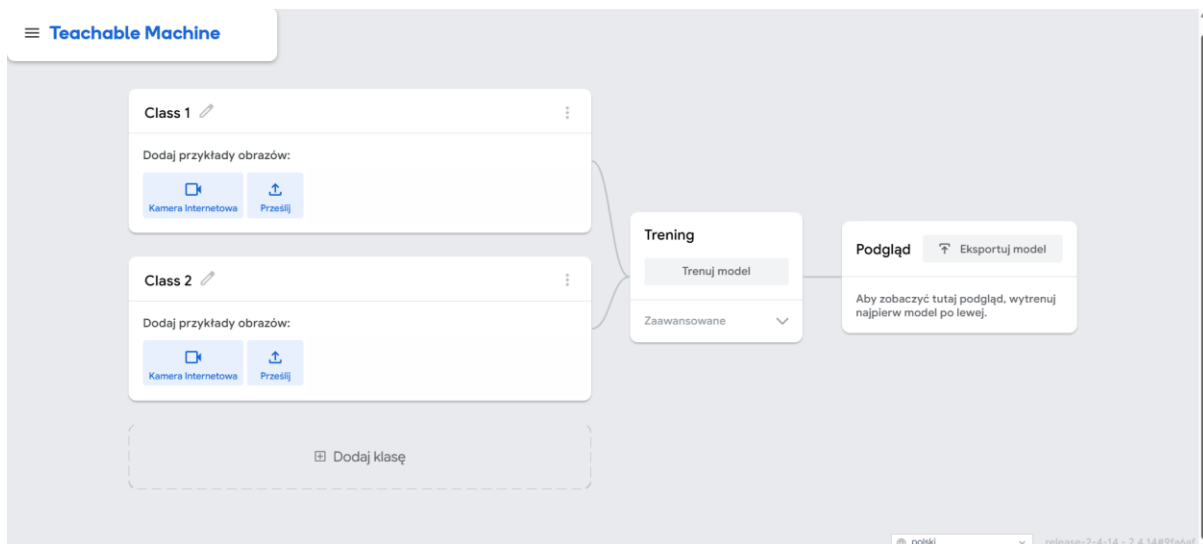


Zostaniesz przekierowany do miejsca na komputerze z przechowywanym folderem. My będziemy go traktowali jedynie jako “przechowywalnię” dla naszych zdjęć. Przy tworzeniu folderu na konkretne klasy najlepiej przekopiować te zdjęcia (trzymając wciśnięty prawy przycisk myszy obejmij niebieską ramką wszystkie zdjęcia; gdy to zrobisz puść prawy przycisk myszy i naciśnij kopiuj) i wkleić je do osobnego, nowego folderu (wpierw musisz stworzyć folder w dowolnym miejscu i odpowiednio go nazwać; zrobiwszy to wklej zdjęcia). Przy okazji będziemy mogli przejrzeć zrobione zdjęcia i odrzucić te niespełniające naszych wymagań.

UWAGA: Warto pilnować, aby w folderze nie było za dużo zdjęć. 200-300 jest w zupełni wystarczające. Zbyt duża liczba przekazywanych zdjęć do Teachable Machines, może powodować problemy na późniejszych etapach.

Problemem tworzenia wielu folderów, a więc i wielu klas zajmiemy się później. Narazie przetrenujemy działanie nauczania komputerowego dla jednej klasy (na bazie nagranych zdjęć podczas testów programu MIT Processing).

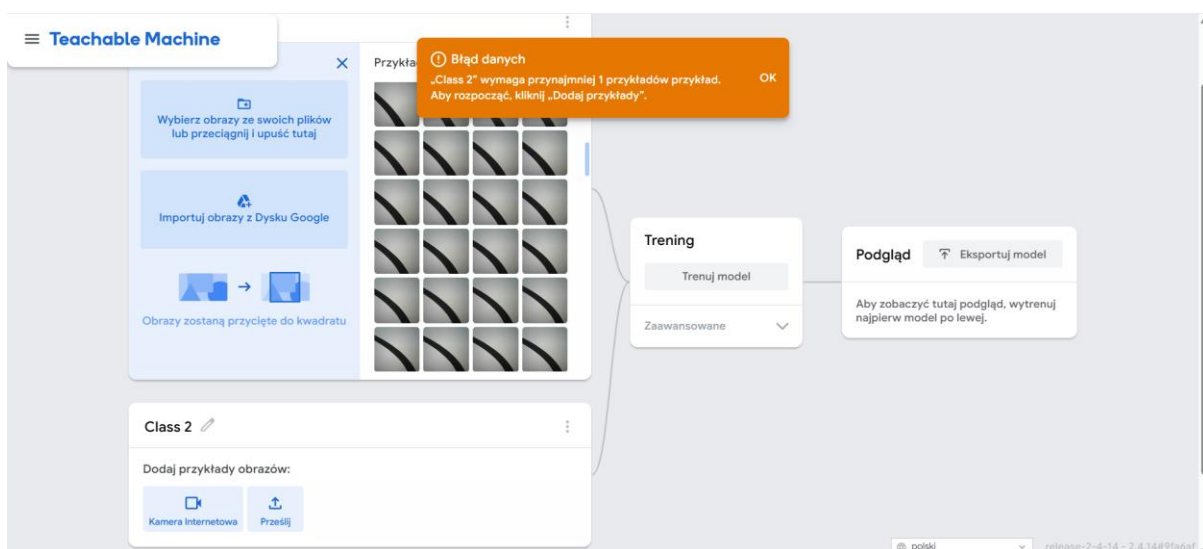
Otwórz stronę [Teachable Machines](#)



Otworzył Ci się panel z opcjami do tworzenia klas. Jeżeli chcesz zmienić nazwę klasy (aby była czytelniejsza) naciśnij *ołówkę* przy nazwie Class 1 i zmień nazwę. Jeżeli chcesz dodać nową klasę naciśnij na dole *Dodaj klasę*. Aby wgrać folder ze zdjęciami naciśnij przycisk *Prześlij* i wybierz odpowiedni folder (w tym przypadku nasz testowy). Następnie *Wybierz obrazy ze swoich plików* i odszukaj swój folder lub też przeciągnij go w to pole.

UWAGA: Prawdopodobnie będziesz musiał zaznaczyć zdjęcia z folderu aby je wkleić do klasy. Postąp dokładnie tak samo jak przy kopiowaniu zdjęć.

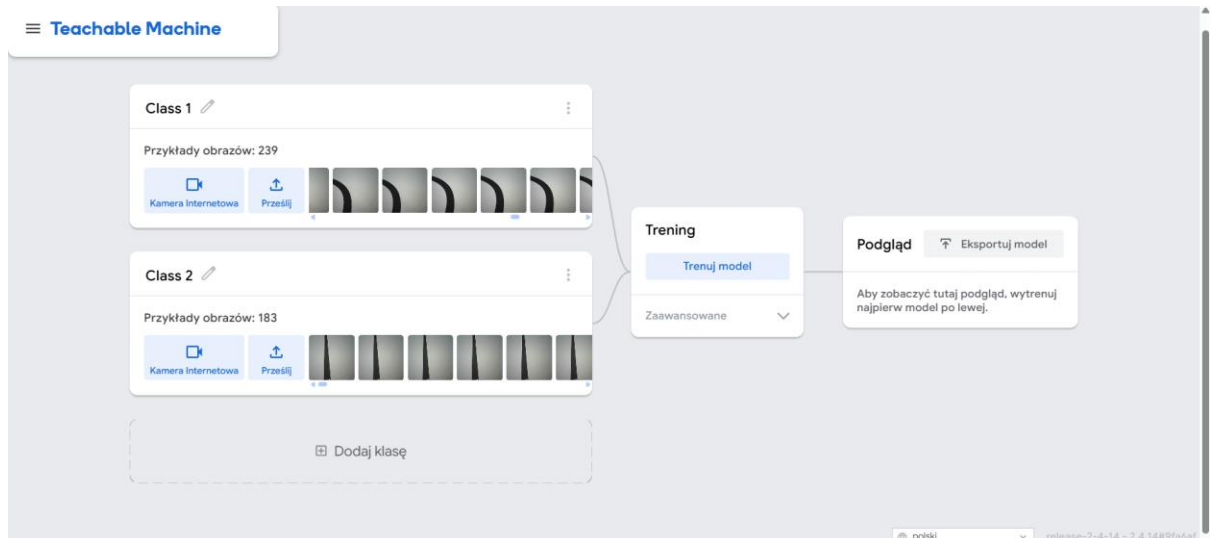
Po wgraniu zdjęć do klasy, jeżeli próbowałeś nacisnąć przycisk *Trenuj model* wyskoczył Ci taki komunikat:



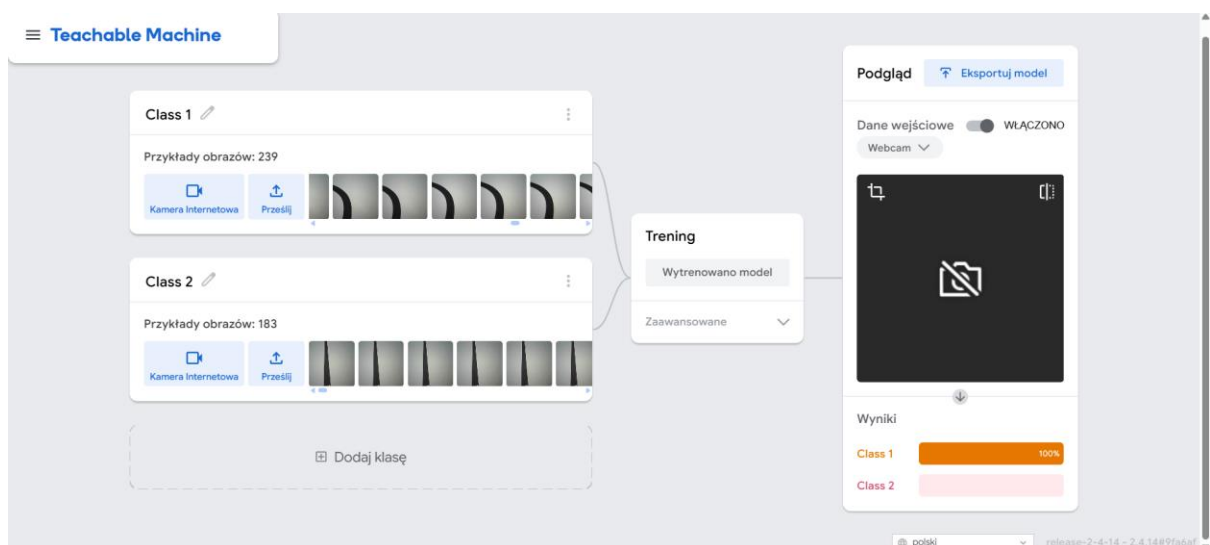
Aby trenować model potrzebna jest co najmniej jeszcze jedna klasa. W końcu program podczas działania ma rozpoznawać klasę spośród paru. Twoim zadaniem jest stworzenie drugiej klasy! Postępuj dokładnie tak samo jak do tej pory. Wcześniej usuń

zdjęcia z folderu, aby zrobić miejsce na następne. Nowy pakiet zdjęć wgraj do Klasy 2. Postaraj się, aby zdjęcia różniły się od tych już wgranych.

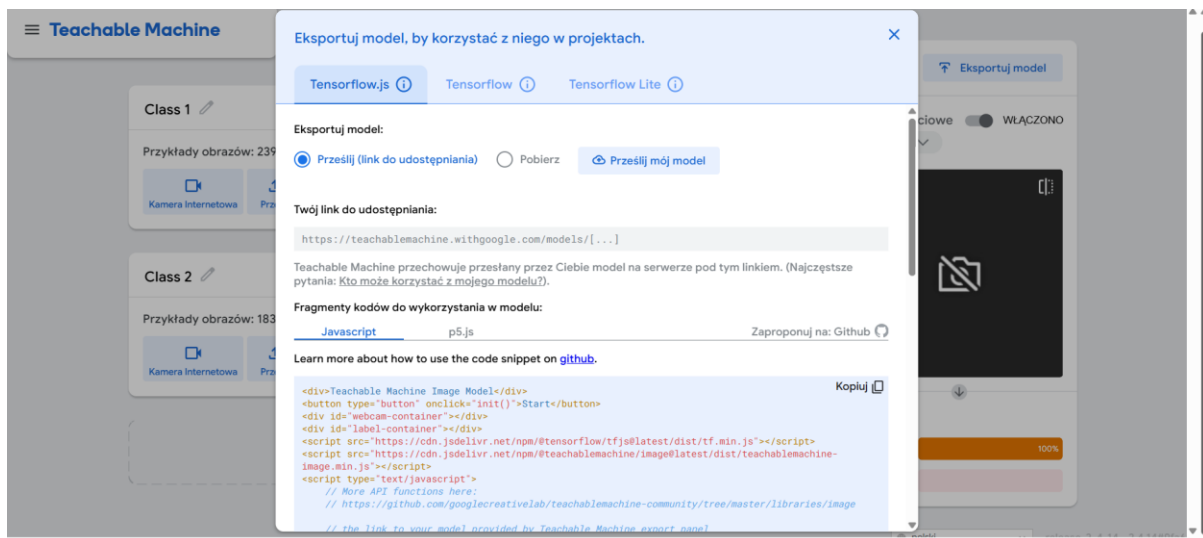
Jeżeli dodałeś drugą klasę to zauważyłeś, że przycisk Trenuj model zmienił kolor na niebieski. Wciśnij go.



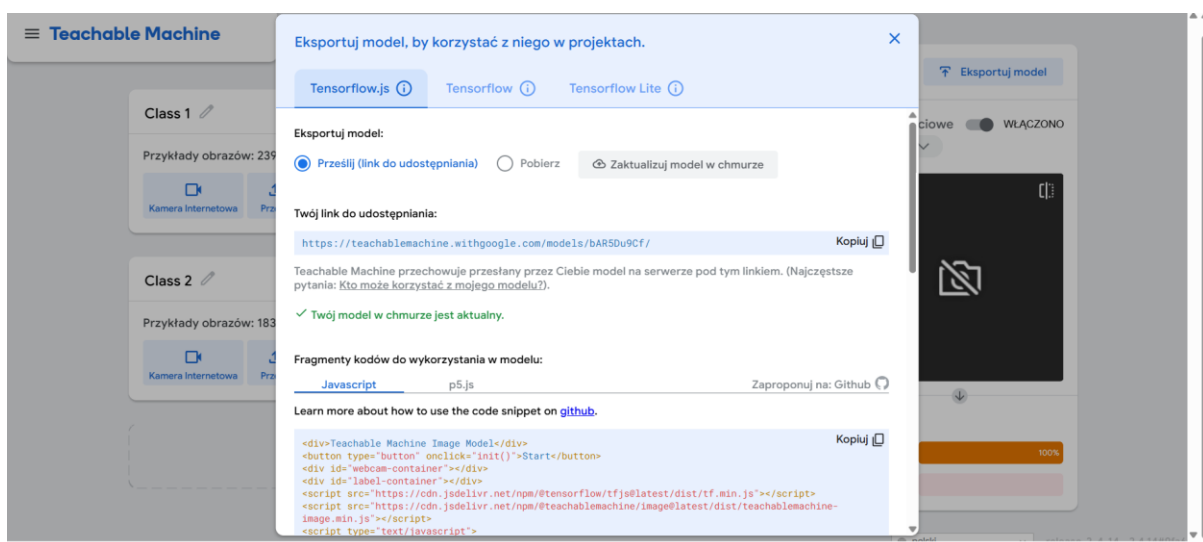
Model zacznie się trenować. Może to chwilę zająć. Ważne jest, aby nie przetaczać okna strony! Po pomyślnym treningu modelu to pojawi Ci się na ekranie:



Gratulacje! Właśnie przetrenowałeś swój pierwszy model do rozpoznawania klas. Teraz program będzie w stanie na podstawie bieżącego obrazu z kamery przyporządkować go do odpowiedniej klasy (rozpoznanej na podstawie podobieństwa do zdjęć z bazy). Prawdopodobnie włączyła Ci się kamera. Nie przejmuj się nią. Nas będzie interesował przycisk *Eksportuj model* (aby móc zaimplementować nasz model do rzeczywiście działającego programu).



Wcisnąwszy ten przycisk pojawi się takie okno. Upewnij się, że na górnym pasku masz wybraną zakładkę *Tensorflow.js*, oraz opcję *Prześlij (link do udostępniania)*. Następnie wybierz opcję *Prześlij mój model*.



Zostanie utworzony link, który należy skopiować.

To była ciężka droga, ale została nam już ostatnia prosta. Pamiętaj stronę internetową, którą otwieraliśmy wklejając adres STAIP (tę z czarnym tłem)?



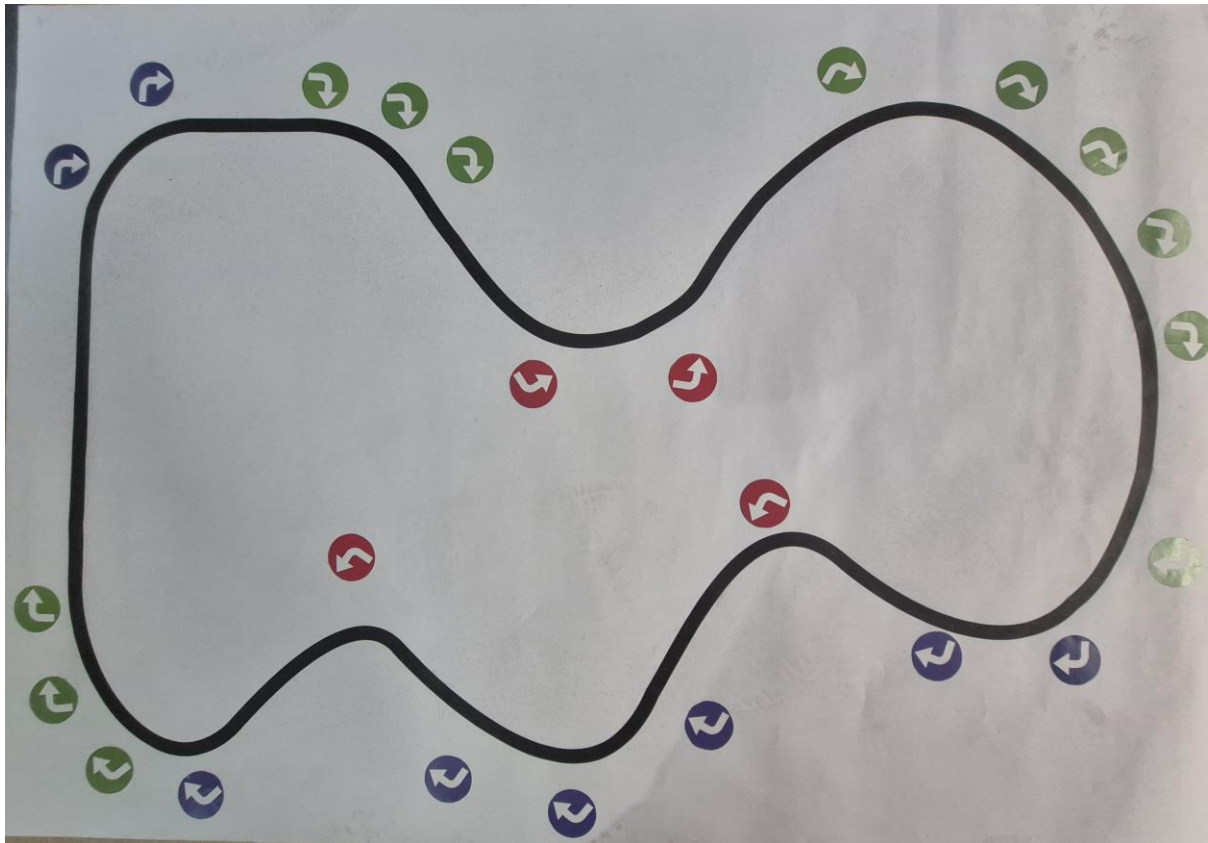
W pasku *Model Path* wklej wygenerowany wcześniej link. Następnie naciśnij przycisk *Start Recognition* i gotowe! Robot mając odpaloną kamerę (której obraz będzie widoczny) będzie rozpoznawał klasy. Zwróć uwagę, że na dole ekranu pojawiły się w rzędach nazwy klas oraz obok nich wartości liczbowe. Wskazują one, w jakim stopniu rozpoznawana jest dana klasa. Suma wszystkich wartości powinna sumować się do 1. Klasa z największym stopniem rozpoznawalności zostanie wyświetlona na dole, a odpowiednia komenda w kodzie zostanie zrealizowana. Przetestuj swojego robota tak, aby mógł rozpoznać wprowadzone przez Ciebie klasy.

Gratulacje! Właśnie zostałeś przeszkolony z używania modeli i klas do nauczania maszynowego. Dobra robota!

ZADANIE

Mając już podstawy teoretyczne, możemy przejść do realizacji zadania. Z pewnością kojarzysz roboty biorące udział w zawodach, które podążając za czarną linią muszą dojechać do celu w jak najkrótszym czasie. Zajmiemy się podobnym zagadnieniem, lecz naszym celem (przynajmniej narazie) nie będzie pobijanie rekordów czasowych, lecz zwykła nauka i dalsza zabawa z nauczaniem maszynowym.

Na płaskiej powierzchni rozłóż tor do jazdy (format A0). Wygląda on tak:



Jest to zamknięta krzywa, mająca wiele względnie łagodnych łuków, jak również fragmenty prostej drogi. Każdy łuk jest opatrzony paroma znakami różnych kolorów: Do skrętu w lewo lub w prawo. Dokładne oznaczenia znaków to:



- łagodny skręt w prawo



- skręt w prawo



- skręt w lewo

Znaki te mają pomóc w stworzeniu klas.

Celem tego zadania jest stworzenie pakietu klas tak, aby robot mógł w miarę bezbłędnie przejechać całą trasę bez spuszczenia z obrazu czarnej linii. Kamera jest tak ustawiona, aby dokładnie widziała czarną linię jak i przestrzeń przed nią.

Jak się zabrać do tego zadania? Na pewno etapami. Warto wpierw zastanowić się ilu i jakich klas będziemy potrzebowali. Chwilę zastanów się nad tym, odpowiedź masz poniżej.

Podpowiedź: Zadanie to zostało testowane dla 6 klas. Dlaczego aż tyle? (Albo tylko tyle?). Następujące klasy to:

Klasa 1 – jazda wprzód (zwykłe podążanie za linią, gdy jest ona na wprost)

Klasa 2 – zjazd do prawej (celem jest korekcja toru, gdy robot znajduje się już po lewej poza linią. Mówimy o przypadku, gdy czarna prosta linia znajduje się po prawej stronie, mniej więcej w 1/3 szerokości obrazu. Robot musi wówczas wykonać bardzo lekki skręt w prawo, aby powrócić na swój tor)

Klasa 3 – zjazd do lewej (analogiczna sytuacja do poprzedniej)

Klasa 4 – skręt w prawo (Robot widząc zakręt oraz znak, musi wykonać skręt w prawo, aby podążać za linią po odchylenie toru)

Klasa 5 - lekki skręt w prawo (Robot widząc zakręt oraz znak, musi wykonać lekki skręt w prawo, aby podążać za linią po niewielkim odchyleniu toru)

Klasa 6 – skręt w lewo (Analogiczna sytuacja do poprzedniej)

Każdej klasie w kodzie (linie ok 400) trzeba dodać jej odpowiednią komendę - funkcję wraz z parametrem. Ponownie zastanów się, jakie polecenia mają być wykonywane dla każdej klasy (spójrz na naszą bibliotekę funkcji). Poniżej masz odpowiedź.

Klasa 1 – `move_Foward(183);`

Klasa 2 – `TurnRight(183);`

Klasa 3 – `TurnLeft(183);`

Klasa 4 - `TurnHardRight(183);`

Klasa 5 - `TurnMidiumRight(183);`

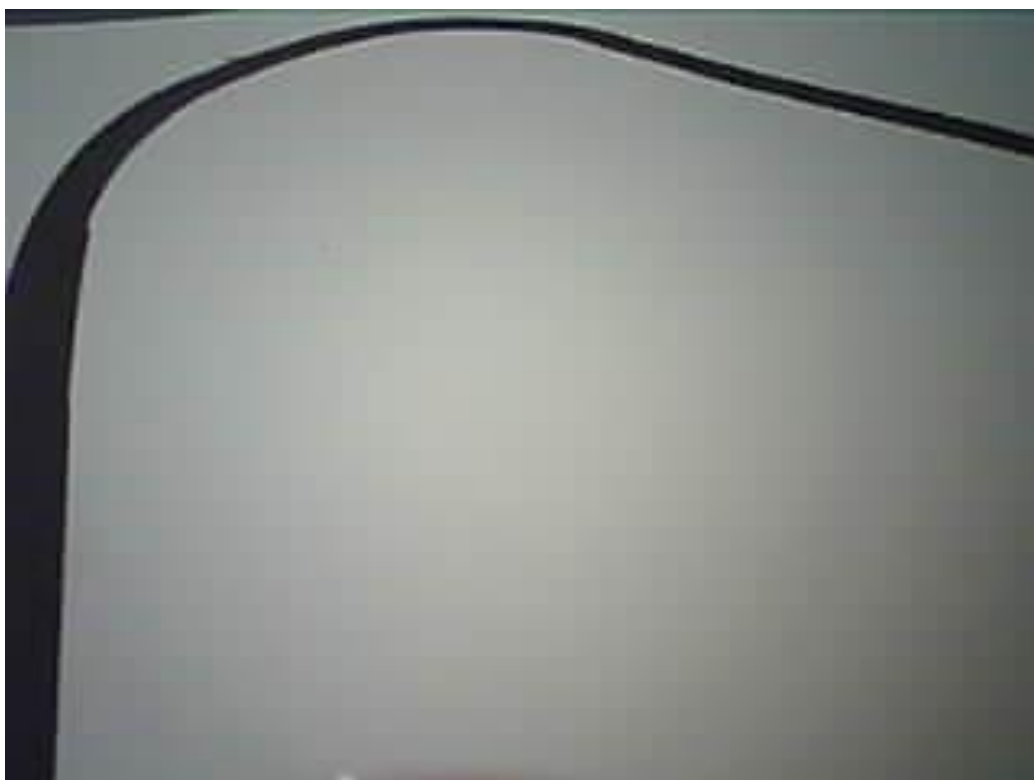
Klasa 6 - `TurnMidiumLeft(183);`

Pamiętaj, że dla każdej funkcji musisz dobrać w nawiasie odpowiednie parametry, określające chociażby czas trwania czy prędkość. Możesz też pokusić się o napisanie własnych funkcji. Gotowy kod do programu znajdziesz w plikach.

Warto rozpocząć zabawę od puszczenia robota na prostej linii i zajęciem się klasami 1,2,3. Robiąc zdjęcia dla poszczególnych klas, trzeba mieć na uwadze, że mogą one zostać nieco przycięte w Teachable Machines. Poniżej znajdują się przykłady dobrych i złych zdjęć:



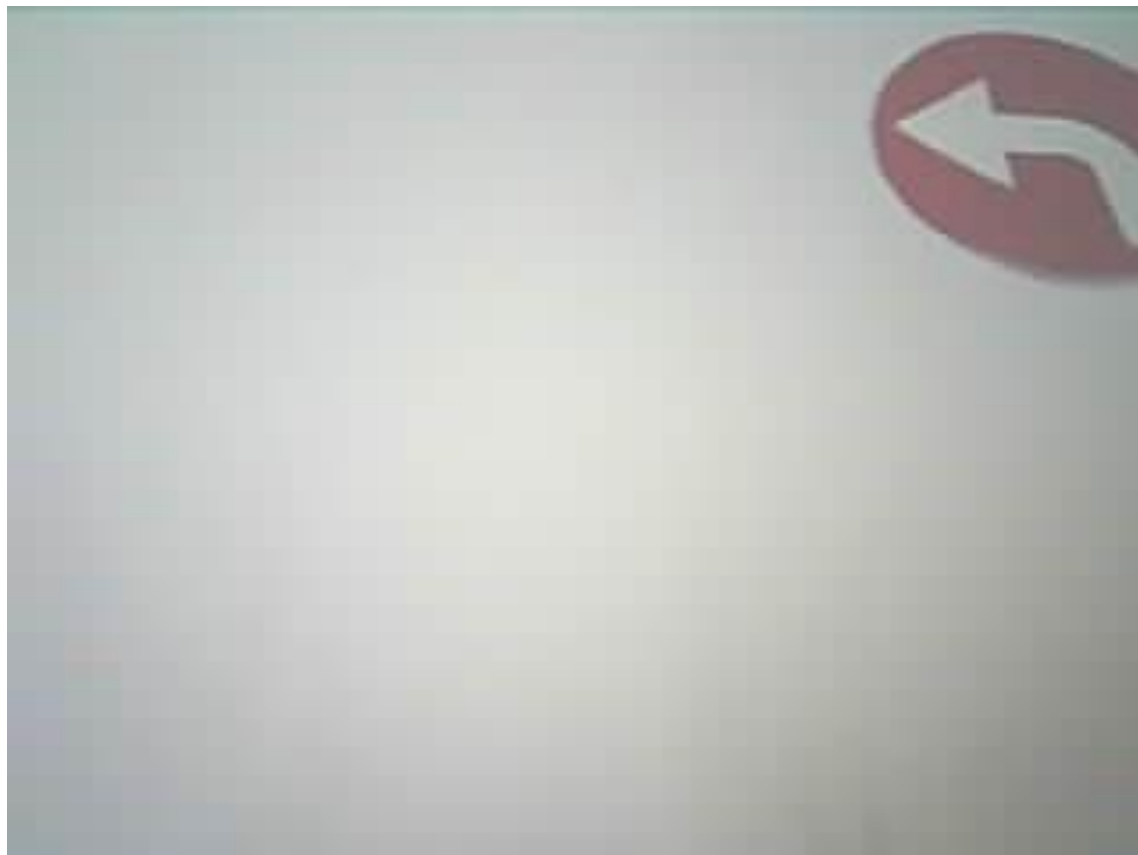
Złe: Jeżeli zdjęcie zostanie ścięte, to czarne paski zostaną ucięte. Program będzie przyporządkowywał białą kartkę jako klasę 3, nawet gdy wyjedzie za linię



Złe: Na górze zdjęcia widoczna jest dalsza linia skręcającego toru. Robot może zinterpretować każdy zakręt w kształcie litery U, jako klasę 3. Jest to zupełnie sprzeczne z ideą klasy 3.



Dobre: Oddana jest istota rzeczy. Widać wyraźnie, że linia znajduje się po lewej stronie, a więc robot ma podstawę, aby wykonać klasę 3. Nawet po lekkim ścięciu zdjęcia linia będzie widoczna



Dobre: Zdjęcie zrobione z oddali i pod kątem, a nie tylko na wprost. Dzięki temu robot będąc nawet inaczej ustawionym będzie w stanie wykonać klasę 6. Na zdjęciu widać, iż jest to kolor czerwony.

Mając już przejrzone powyższe przykłady, postaraj się sam zrobić zdjęcia do klas. Za każdym razem musisz zdjęcia z folderu przekopiować do odpowiedniej klasy w Teachable Machines i po stworzeniu paru klas przetrenować model i pobrać wygenerowany link. Testuj jazdę robota i w razie niepowodzeń (robot wyjeżdża za linię, myli klasy), zrób nowy zestaw zdjęć. Możesz każdy zakręt na torze sfotografować i wkleić do odpowiedniej klasy, chociaż nawet dla jednego, dobrze sfotografowanego zakrętu robot będzie sobie radził z pozostałymi. Może to być żmudne zajęcie, gdyż nawet drobne zmiany w jednej klasie będą wymagały przetrenowania całego modelu od nowa. W przypadku dużych problemów możesz pobrać nasz folder z gotowymi zdjęciami. Wystarczy, że wstawisz ten folder do Teachable Machines.

Podobnie postąp dla kolejnych zakrętów i klas. Gdy robot przejedzie całą trasę bez większych problemów gratulacje, wykonałeś kawał porządnej roboty. Jeżeli chcesz możesz potestować działanie robota dla jego szybszej prędkości lub też zmodyfikować klasy. Baw się dobrze!

Pomysły na przyszłość: rozpoznawanie znaków drogowych, skrzyżowania, rozwidlenia dróg, ostrzejsze zakręty, detektor linii, computer vision (color recognition) i wiele więcej

MATERIAŁY

Złożony Robot Sharky z kamerą esp32 oraz wydrukowany na drukarce 3D trzymak do kamery, komputer PC z Arduino IDE 2.3.8, szablon A0 z gotowym torem jazdy, powerbank Esperanza

POMOC NAUCZYCIELA

Nauczyciel pełni rolę obserwatora, patrząc na poprawność wykonywania poleceń z instrukcji przez uczniów. Instrukcje są napisane krok po kroku, więc nie powinno być problemów z jej wykonaniem. Głównym obowiązkiem prowadzącego jest dbanie o bezpieczeństwo uczniów w trakcie trwania zajęć oraz ewentualna pomoc przy pobieraniu programów, tworzeniu folderów, programowaniu. Nauczyciel nadzoruje pracę z urządzeniami, dbając o nieniszczenie sprzętu (komputerów, zmontowanych robotów Sharky oraz powerbanków).

WYKORZYSTANIE ROBOTA Z KAMERĄ ESP32 W NAUCE O AI

Robot służy jako fizyczne narzędzie do zbierania zbioru danych – dzięki modułowi kamery uczeń może na bieżąco przechwytywać widok trasy w formie zdjęć, stanowiących fundament dla sztucznej inteligencji. Po stworzeniu w Teachable Machine odpowiednich klas (np. widok konkretnego znaku lub odpowiedniego odchylenia linii) i wytrenowaniu modelu, program wykonuje adekwatne polecenia w czasie rzeczywistym. Dzięki pracy z urządzeniem, uczeń widzi, jak oprogramowanie oparte o eksport modelu AI komunikuje się ze sprzętem za pośrednictwem sieci Wi-Fi i warunkuje zachowanie silników robota.

PRZEWIDYWANE EFEKTY UCZENIA

Uczeń potrafi otworzyć, analizować i modyfikować istniejący kod w środowisku Arduino IDE (m.in. zmiana zmiennych, konfigurowanie logiki w instrukcjach warunkowych *if / else if*). Uczeń potrafi skonfigurować środowisko MIT Processing w celu przechwytywania i archiwizowania obrazu z kamery IP. Uczeń rozumie pojęcie uczenia maszynowego na podstawie obrazu, potrafi odpowiednio przygotować zestaw danych uczących i odpowiednio kategoryzować je na wyznaczone "klasy". Uczeń potrafi wygenerować (przetrenować) oraz wyeksportować gotowy model w narzędziu Teachable Machine, a następnie poprawnie wdrożyć go w interfejsie testowym robota do wykonywania określonych zadań na torze.